

Automation Test Scenarios

by Nishith Vadher on 22-May-26 for Interactive Investor Assignment

Assignment Website URL - <https://automationexercise.com>

I have created basic framework in **Playwright** using **Typescript** as language for Automation Regression testing for the given website. Additionally, I have design and develop Four Automation Test scripts for below selected journeys.

Please find the answers for below questions -

- Can you review the website and pick six most important user journeys from the site?

Answer –

1	Login Journey
2	Logout Journey
3	Search Product Journey
4	View Product, Add to Cart and View Cart Journey
5	Complete Shopping Journey
6	User Registration Journey
7	Search Category and Brand Journey

- Out of the six user journeys proceed to pick four journeys to script against and explain why you picked these journeys?

Answer –

1	Login Journey	Scripting Done
2	Logout Journey	Scripting Done
3	Search Product Journey	Scripting Done
4	View Product, Add to Cart and View Cart Journey	Scripting Done

I have selected first three journeys, which are very important for regression testing as first any User should able to do **Login** on the website and then user should able to **Logout**. After that any user should able to **search the product** for shopping and should able to **view product details**. After viewing product, if user like the product, then should able to **add the product to cart (add to cart)** and further adding the product to cart, user should **view the cart**. Once user has added all the required products into the cart then proceed to checkout and if not logged in do login or register the website and do final payment and complete the shopping.

- Build a basic framework from scratch and script the four journeys you have chosen previously. Languages should be Typescript (preferable) or JavaScript, and making use of Webdriver.io (preferable) or Playwright to match our current technologies.

Answer –

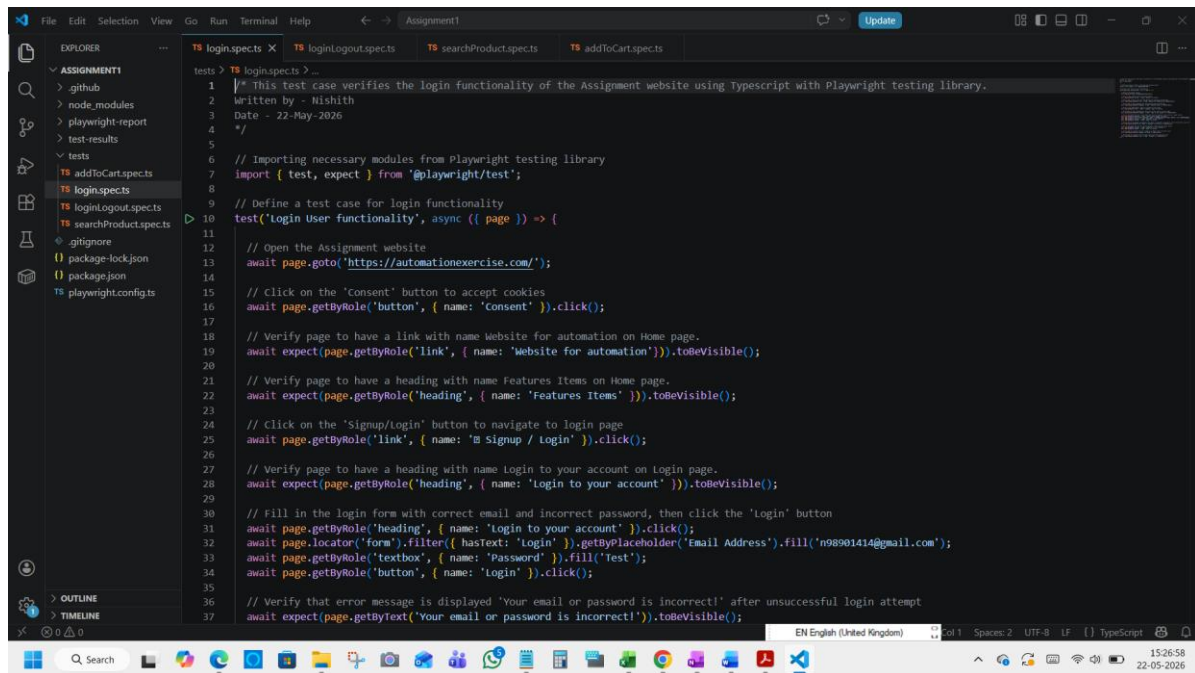
I have taken **first four journeys** from the **above table** and developed automation test scripts from scratch. I have created a **basic automation framework** in **Playwright**, using **Typescript** as language. All the four journeys are designed, developed and executed in **Chromium** browser. Reports are also generated and four test scripts were passed, adding few screenshots for reference -

Note:

- 1> I have also covered a **Negative scenario** for **incorrect password** while testing **Login functionality** script.
- 2> I have provided comments for most of the lines for better understanding the code and its purpose.
- 3> I have used **MS-VS code** for writing the scripts and done execution for **only Chromium browser** for **Playwright framework** and **Typescript language**.

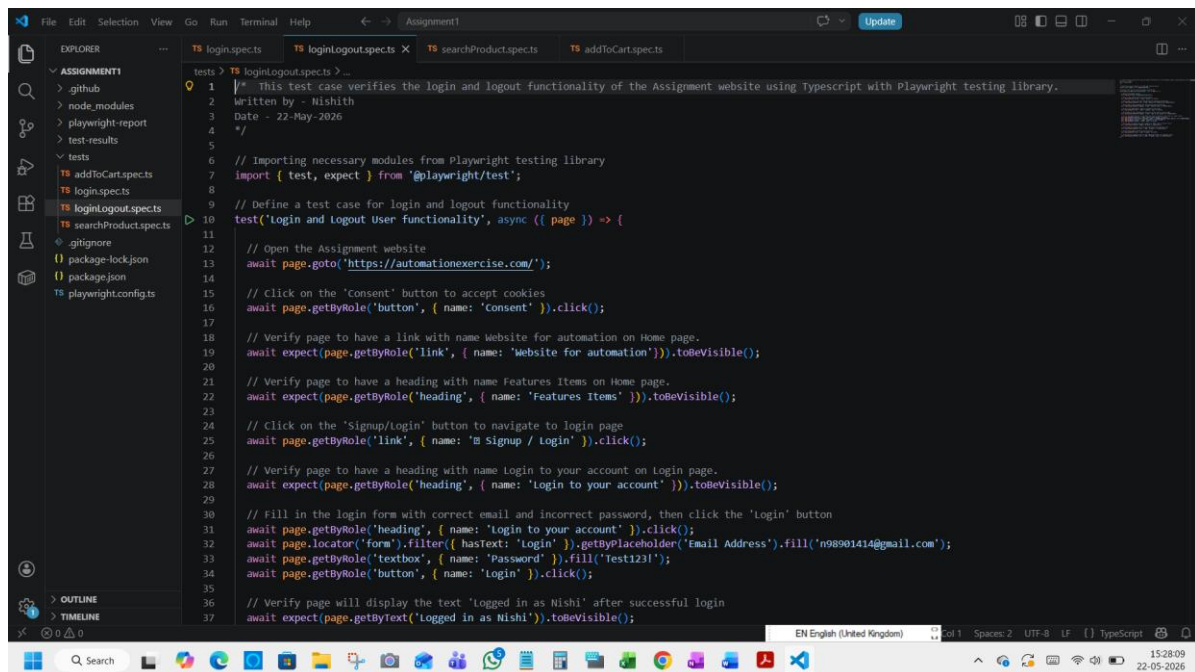
Few Screenshots –

1> Login.spec.ts



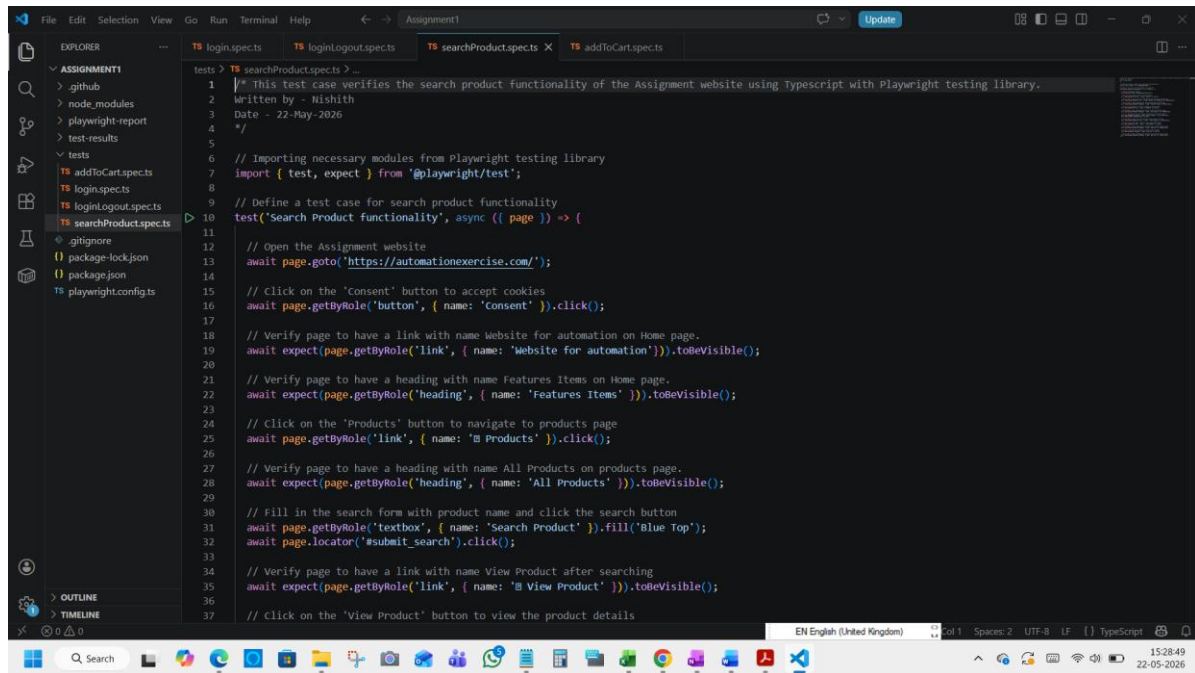
```
1  |/* This test case verifies the login functionality of the Assignment website using Typescript with Playwright testing library.
2  |Written by - Nishith
3  |Date - 22-May-2026
4  |*/
5
6  // Importing necessary modules from Playwright testing library
7  import { test, expect } from '@playwright/test';
8
9  // Define a test case for login functionality
10 test('Login User functionality', async ({ page }) => {
11
12     // Open the Assignment website
13     await page.goto('https://automationexercise.com/');
14
15     // click on the 'consent' button to accept cookies
16     await page.getByRole('button', { name: 'Consent' }).click();
17
18     // Verify page to have a link with name Website for automation on Home page.
19     await expect(page.getByRole('link', { name: 'Website for automation' })).toBeVisible();
20
21     // Verify page to have a heading with name Features Items on Home page.
22     await expect(page.getByRole('heading', { name: 'Features Items' })).toBeVisible();
23
24     // Click on the 'Signup/Login' button to navigate to login page
25     await page.getByRole('link', { name: 'B Signup / Login' }).click();
26
27     // Verify page to have a heading with name login to your account on login page.
28     await expect(page.getByRole('heading', { name: 'login to your account' })).toBeVisible();
29
30     // Fill in the login form with correct email and incorrect password, then click the 'login' button
31     await page.getByRole('heading', { name: 'login to your account' }).click();
32     await page.locator('form').filter({ hasText: 'Login' }).getByPlaceholder('Email Address').fill('n98901414@gmail.com');
33     await page.getByRole('text', { name: 'Password' }).fill('Test');
34     await page.getByRole('button', { name: 'login' }).click();
35
36     // Verify that error message is displayed 'Your email or password is incorrect!' after unsuccessful login attempt
37     await expect(page.getByText('Your email or password is incorrect!')).toBeVisible();
38 }
```

2> loginLogout.spec.ts



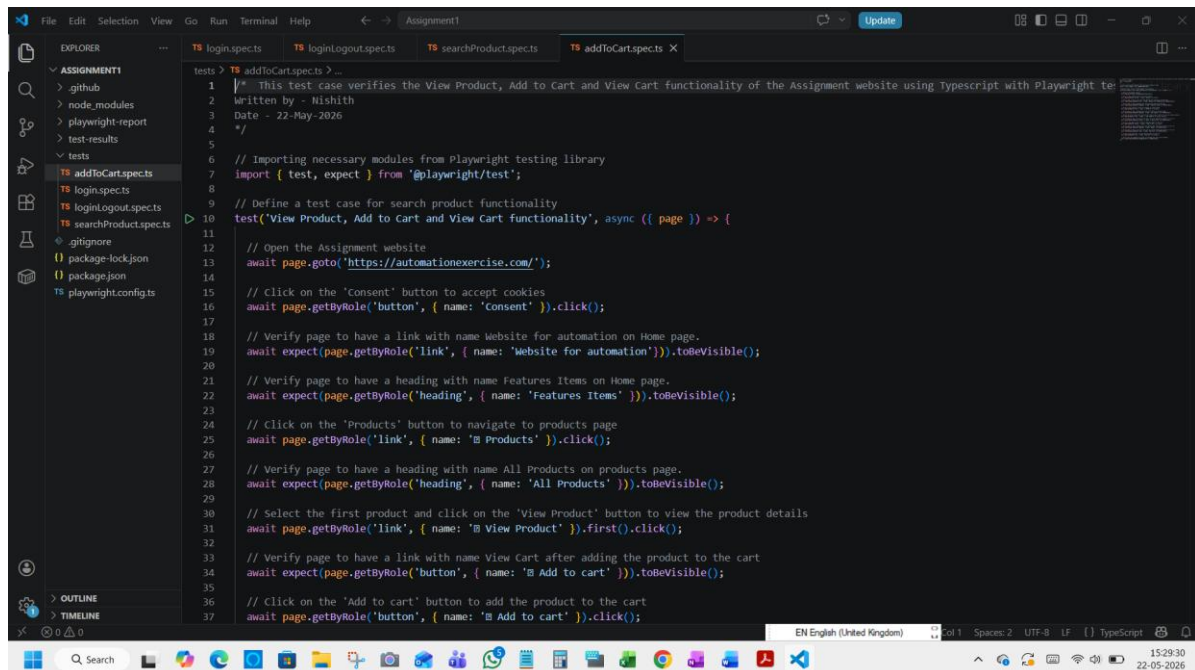
```
1  |/* This test case verifies the login and logout functionality of the Assignment website using Typescript with Playwright testing library.
2  |Written by - Nishith
3  |Date - 22-May-2026
4  |*/
5
6  // Importing necessary modules from Playwright testing library
7  import { test, expect } from '@playwright/test';
8
9  // Define a test case for login and logout functionality
10 test('Login and Logout User functionality', async ({ page }) => {
11
12     // Open the Assignment website
13     await page.goto('https://automationexercise.com/');
14
15     // click on the 'consent' button to accept cookies
16     await page.getByRole('button', { name: 'Consent' }).click();
17
18     // Verify page to have a link with name Website for automation on Home page.
19     await expect(page.getByRole('link', { name: 'Website for automation' })).toBeVisible();
20
21     // Verify page to have a heading with name Features Items on Home page.
22     await expect(page.getByRole('heading', { name: 'Features Items' })).toBeVisible();
23
24     // Click on the 'Signup/Login' button to navigate to login page
25     await page.getByRole('link', { name: 'B Signup / Login' }).click();
26
27     // Verify page to have a heading with name login to your account on login page.
28     await expect(page.getByRole('heading', { name: 'login to your account' })).toBeVisible();
29
30     // Fill in the login form with correct email and incorrect password, then click the 'login' button
31     await page.getByRole('heading', { name: 'login to your account' }).click();
32     await page.locator('form').filter({ hasText: 'Login' }).getByPlaceholder('Email Address').fill('n98901414@gmail.com');
33     await page.getByRole('text', { name: 'Password' }).fill('Test123!');
34     await page.getByRole('button', { name: 'login' }).click();
35
36     // Verify page will display the text 'logged in as Nishi' after successful login
37     await expect(page.getByText('logged in as Nishi')).toBeVisible();
38 }
```

3> searchProduct.spec.ts



```
1  /* This test case verifies the search product functionality of the Assignment website using Typescript with Playwright testing library.
2  Written by - Nishith
3  Date - 22-May-2026
4  */
5
6  // Importing necessary modules from Playwright testing library
7  import { test, expect } from '@playwright/test';
8
9  // Define a test case for search product functionality
10 test('Search Product functionality', async ({ page }) => {
11
12     // Open the Assignment website
13     await page.goto('https://automationexercise.com/');
14
15     // Click on the 'consent' button to accept cookies
16     await page.getByRole('button', { name: 'Consent' }).click();
17
18     // Verify page to have a link with name Website for automation on Home page.
19     await expect(page.getByRole('link', { name: 'Website for automation' })).toBeVisible();
20
21     // Verify page to have a heading with name Features Items on Home page.
22     await expect(page.getByRole('heading', { name: 'Features Items' })).toBeVisible();
23
24     // Click on the 'Products' button to navigate to products page
25     await page.getByRole('link', { name: 'B Products' }).click();
26
27     // Verify page to have a heading with name All Products on products page.
28     await expect(page.getByRole('heading', { name: 'All Products' })).toBeVisible();
29
30     // Fill in the search form with product name and click the search button
31     await page.getByRole('textbox', { name: 'Search Product' }).fill('Blue Top');
32     await page.locator('#submit_search').click();
33
34     // Verify page to have a link with name View Product after searching
35     await expect(page.getByRole('link', { name: 'B View Product' })).toBeVisible();
36
37     // Click on the 'View Product' button to view the product details
```

4> addToCart.spec.ts



```
1  /* This test case verifies the View Product, Add to Cart and View Cart functionality of the Assignment website using Typescript with Playwright testing library.
2  Written by - Nishith
3  Date - 22-May-2026
4  */
5
6  // Importing necessary modules from Playwright testing library
7  import { test, expect } from '@playwright/test';
8
9  // Define a test case for search product functionality
10 test('View Product, Add to cart and View Cart functionality', async ({ page }) => {
11
12     // Open the Assignment website
13     await page.goto('https://automationexercise.com/');
14
15     // Click on the 'consent' button to accept cookies
16     await page.getByRole('button', { name: 'Consent' }).click();
17
18     // Verify page to have a link with name Website for automation on Home page.
19     await expect(page.getByRole('link', { name: 'Website for automation' })).toBeVisible();
20
21     // Verify page to have a heading with name Features Items on Home page.
22     await expect(page.getByRole('heading', { name: 'Features Items' })).toBeVisible();
23
24     // Click on the 'Products' button to navigate to products page
25     await page.getByRole('link', { name: 'B Products' }).click();
26
27     // Verify page to have a heading with name All Products on products page.
28     await expect(page.getByRole('heading', { name: 'All Products' })).toBeVisible();
29
30     // Select the first product and click on the 'View Product' button to view the product details
31     await page.getByRole('link', { name: 'B View Product' }).first().click();
32
33     // Verify page to have a link with name View Cart after adding the product to the cart
34     await expect(page.getByRole('button', { name: 'B Add to cart' })).toBeVisible();
35
36     // Click on the 'Add to cart' button to add the product to the cart
37     await page.getByRole('button', { name: 'B Add to cart' }).click();
```

5> FourScript – Execution Report

The screenshot displays the FourScript execution report for the 'chromium' project. The interface includes a search bar, a filter dropdown set to 'All', and a summary of test results: 4 Passed, 4 Failed, 0 Flaky, and 0 Skipped. The total time for the tests is 13.8s, recorded on 22/05/2026 at 13:31:29. The tests are listed in a table with their names and durations:

Test Name	Duration
addToCart.spec.ts	
✓ View Product, Add to Cart and View Cart functionality (chromium)	10.5s
login.spec.ts	
✓ Login User functionality (chromium)	9.9s
loginLogout.spec.ts	
✓ Login and Logout User functionality (chromium)	11.6s
searchProduct.spec.ts	
✓ Search Product functionality (chromium)	9.0s

The bottom of the screenshot shows the Windows taskbar with the system clock at 15:30:36 on 22-05-2026.

Individual Report-

The screenshot shows the individual test report for the 'View Product, Add to Cart and View Cart functionality' test. The test is identified by the ID 'addToCart.spec.ts:10' and has a duration of 10.5s. The 'Test Steps' section provides a detailed breakdown of the test execution:

Test Step	Duration
Before Hooks	1.1s
✓ Navigate to '/' — addToCart.spec.ts:13	3.6s
✓ Click getByRole('button', { name: 'Consent' }) — addToCart.spec.ts:16	312ms
✓ Expect 'toBeVisible' getByRole('link', { name: 'Website for automation' }) — addToCart.spec.ts:19	35ms
✓ Expect 'toBeVisible' getByRole('heading', { name: 'Features Items' }) — addToCart.spec.ts:22	17ms
✓ Click getByRole('link', { name: 'Products' }) — addToCart.spec.ts:25	924ms
✓ Expect 'toBeVisible' getByRole('heading', { name: 'All Products' }) — addToCart.spec.ts:28	79ms
✓ Click getByRole('link', { name: 'View Product' }) [first] — addToCart.spec.ts:31	1.6s
✓ Expect 'toBeVisible' getByRole('button', { name: 'Add to cart' }) — addToCart.spec.ts:34	115ms
✓ Click getByRole('button', { name: 'Add to cart' }) — addToCart.spec.ts:37	260ms
✓ Expect 'toBeVisible' getByRole('heading', { name: 'Added!' }) — addToCart.spec.ts:40	1.4s
✓ Expect 'toBeVisible' getByRole('link', { name: 'View Cart' }) — addToCart.spec.ts:43	12ms
✓ Click getByRole('link', { name: 'View Cart' }) — addToCart.spec.ts:46	1.4s
✓ Expect 'toBeVisible' getByText('Shopping Cart') — addToCart.spec.ts:49	44ms
After Hooks	24ms

The bottom of the screenshot shows the Windows taskbar with the system clock at 15:36:47 on 22-05-2026.

localhost:9323/#?testId=41d3fd2474a19feb00a1-128e34a345b715a6d0ff

Search tests

All 4 ✓ Passed 4 Failed 0 Flaky 0 Skipped 0

Login User functionality

login.spec.ts:10 9.9s

Run

Test Steps

Filter steps

- Before Hooks 1.0s
- Navigate to "/" — login.spec.ts:13 3.5s
- Click getByRole(button, { name: 'Consent' }) — login.spec.ts:16 356ms
- Expect "toBeVisible" getByRole(link, { name: 'Website for automation' }) — login.spec.ts:19 32ms
- Expect "toBeVisible" getByRole(heading, { name: 'Features Items' }) — login.spec.ts:22 28ms
- Click getByRole(link, { name: 'Sign up / Login' }) — login.spec.ts:25 818ms
- Expect "toBeVisible" getByRole(heading, { name: 'Login to your account' }) — login.spec.ts:28 118ms
- Click getByRole(heading, { name: 'Login to your account' }) — login.spec.ts:31 219ms
- Fill "n98901414@gmail.com" locator(form).filter(hasText: 'Login').getByPlaceholder('Email Address') — login.spec.ts:32 48ms
- Fill "Test" getByRole(textbox, { name: 'Password' }) — login.spec.ts:33 29ms
- Click getByRole(button, { name: 'Login' }) — login.spec.ts:34 1.4s
- Expect "toBeVisible" getByText('Your email or password is incorrect') — login.spec.ts:37 99ms
- Fill "Test123" getByRole(textbox, { name: 'Password' }) — login.spec.ts:40 35ms
- Click getByRole(button, { name: 'Login' }) — login.spec.ts:41 2.6s
- Expect "toBeVisible" getByText('Logged in as Nish') — login.spec.ts:44 91ms
- Expect "toBeVisible" getByRole(link, { name: 'Logout' }) — login.spec.ts:47 24ms

EN English (United Kingdom) 15:38:28 22-05-2026

localhost:9323/#?testId=a720bdbaebd985f1ea1d-4502e8e14ea7b219f5a6

Search tests

All 4 ✓ Passed 4 Failed 0 Flaky 0 Skipped 0

Login and Logout User functionality

loginLogout.spec.ts:10 11.6s

Run

Test Steps

Filter steps

- Before Hooks 1.2s
- Navigate to "/" — loginLogout.spec.ts:13 3.4s
- Click getByRole(button, { name: 'Consent' }) — loginLogout.spec.ts:16 449ms
- Expect "toBeVisible" getByRole(link, { name: 'Website for automation' }) — loginLogout.spec.ts:19 76ms
- Expect "toBeVisible" getByRole(heading, { name: 'Features Items' }) — loginLogout.spec.ts:22 15ms
- Click getByRole(link, { name: 'Sign up / Login' }) — loginLogout.spec.ts:25 903ms
- Expect "toBeVisible" getByRole(heading, { name: 'Login to your account' }) — loginLogout.spec.ts:28 101ms
- Click getByRole(heading, { name: 'Login to your account' }) — loginLogout.spec.ts:31 246ms
- Fill "n98901414@gmail.com" locator(form).filter(hasText: 'Login').getByPlaceholder('Email Address') — loginLogout.spec.ts:32 28ms
- Fill "Test123" getByRole(textbox, { name: 'Password' }) — loginLogout.spec.ts:33 18ms
- Click getByRole(button, { name: 'Login' }) — loginLogout.spec.ts:34 4.0s
- Expect "toBeVisible" getByText('Logged in as Nish') — loginLogout.spec.ts:37 85ms
- Expect "toBeVisible" getByRole(link, { name: 'Logout' }) — loginLogout.spec.ts:40 23ms
- Click getByRole(link, { name: 'Logout' }) — loginLogout.spec.ts:43 1.5s
- Expect "toBeVisible" getByRole(link, { name: 'Sign up / Login' }) — loginLogout.spec.ts:46 30ms
- After Hooks 21ms

EN English (United Kingdom) 15:39:23 22-05-2026

localhost:9323/#?testId=404f6bd6c4f3c9982e9-fa26d8667a8eb712c81f

zIMPMacCdesAltrinchamMainMain2Main3NVTesting JobsNewMainSpainNewJob2026InterviewFAQsAI

Search testsAll4Passed4Failed0Flaky0Skipped0

Search Product functionality

searchProduct.spec.ts:109.0s

Chromium

Run

Test Steps

Filter steps

Before Hooks1.0s

Navigate to "/7" → searchProduct.spec.ts:133.2s

Click getByRole(button), (name: "Consent") → searchProduct.spec.ts:16576ms

Expect "toBeVisible" getByRole(link), (name: "Website for automation") → searchProduct.spec.ts:1925ms

Expect "toBeVisible" getByRole(heading), (name: "Features Items") → searchProduct.spec.ts:2224ms

Click getByRole(link), (name: "Products") → searchProduct.spec.ts:25977ms

Expect "toBeVisible" getByRole(heading), (name: "All Products") → searchProduct.spec.ts:2897ms

Fill "Blue Top" getByRole(textbox), (name: "Search Product") → searchProduct.spec.ts:31182ms

Click locator(#submit_search) → searchProduct.spec.ts:321.7s

Expect "toBeVisible" getByRole(link), (name: "View Product") → searchProduct.spec.ts:3581ms

Click getByRole(link), (name: "View Product") → searchProduct.spec.ts:381.2s

Expect "toBeVisible" getByRole(heading), (name: "Blue Top") → searchProduct.spec.ts:4187ms

Click getByRole(heading), (name: "Blue Top") → searchProduct.spec.ts:44192ms

Expect "toBeVisible" getByRole(heading), (name: "Blue Top") → searchProduct.spec.ts:4710ms

After Hooks54ms

EN English (United Kingdom)15:40:1022-05-2026